

Nama : Deswita Khansa Rafifah

NIM : 254107020151

Kelas : TI-1G

LAPORAN JOBSHEET 12

12.2 Percobaan 1: Operasi Penambahan pada Double Linked List

→ Kode Program

> Kode program class Mahasiswa06.java

```
package Jobsheet12;

public class Mahasiswa06 {
    String nim;
    String nama;
    String kelas;
    double ipk;

    public Mahasiswa06(String nim, String nama, String kelas, double ipk) {
        this.nim = nim;
        this.nama = nama;
        this.kelas = kelas;
        this.ipk = ipk;
    }

    public void tampil() {
        System.out.println(
            "NIM          : " + nim +
            "\nNama         : " + nama +
            "\nKelas        : " + kelas +
            "\nIPK           : " + ipk
        );
    }
}
```

> Kode program class Node06.java

```
package Jobsheet12;
public class Node06 {
    Mahasiswa06 data;
    Node06 prev;
    Node06 next;

    public Node06(Mahasiswa06 data) {
        this.data = data;
        this.prev = null;
        this.next = null;
    }
}
```

> **Kode Program class DoubleLinkedList06.java**

```
package Jobsheet12;

public class DoubleLinkedList06 {
    Node06 head;
    Node06 tail;

    public DoubleLinkedList06() {
        head = null;
        tail = null;
    }

    public boolean isEmpty() {
        return head == null;
    }

    public void addFirst(Mahasiswa06 data) {
        Node06 newNode = new Node06(data);
        if (isEmpty()) {
            head = tail = newNode;
        } else {
            newNode.next = head;
            head.prev = newNode;
            head = newNode;
        }
    }

    public void addLast(Mahasiswa06 data) {
        Node06 newNode = new Node06(data);
        if (isEmpty()) {
            head = tail = newNode;
        } else {
            tail.next = newNode;
            newNode.prev = tail;
            tail = newNode;
        }
    }

    public void insertAfter(String keyNim, Mahasiswa06 data) {
        Node06 current = head;
        while (current != null && !current.data.nim.equals(keyNim)) {
            current = current.next;
        }
        if (current == null) {
            System.out.println("Data dengan NIM " + keyNim + " tidak ditemukan.");
            return;
        }

        Node06 newNode = new Node06(data);
```

```

        if (current == tail) {
            newNode.prev = current;
            current.next = newNode;
            tail = newNode;
        } else {
            newNode.prev = current;
            newNode.next = current.next;
            current.next.prev = newNode;
            current.next = newNode;
        }
        System.out.println("Data berhasil disisipkan setelah NIM " + keyNim);
    }

    public void print() {
        if (isEmpty()) {
            System.out.println("Linked List masih kosong.");
            return;
        }

        Node06 current = head;
        while (current != null) {
            System.out.println("-----");
            current.data.tampil();
            current = current.next;
        }
    }
}

```

> Kode Program class DoubleLinkedListMain06.java

```

package Jobsheet12;
import java.util.Scanner;

public class DoubleLinkedListMain06 {
    public static Mahasiswa06 inputMahasiswa(Scanner scan) {
        System.out.print("Masukkan NIM : ");
        String nim = scan.nextLine();
        System.out.print("Masukkan Nama : ");
        String nama = scan.nextLine();
        System.out.print("Masukkan Kelas : ");
        String kelas = scan.nextLine();
        System.out.print("Masukkan IPK : ");
        double ipk = scan.nextDouble();
        scan.nextLine();
        return new Mahasiswa06(nim, nama, kelas, ipk);
    }

    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        DoubleLinkedList06 list = new DoubleLinkedList06();
    }
}

```

```

int pilihan;

do {
    System.out.println("\n===== MENU DOUBLE LINKED LIST =====");
    System.out.println("1. Tambah data di awal");
    System.out.println("2. Tambah data di akhir");
    System.out.println("3. Sisipkan data di tengah (setelah NIM)");
    System.out.println("4. Hapus data di awal");
    System.out.println("5. Hapus data di akhir");
    System.out.println("6. Tampilkan data");
    System.out.println("0. Keluar");
    System.out.print("Pilih menu : ");
    pilihan = scan.nextInt();
    scan.nextLine();

    switch (pilihan) {
        case 1:
            Mahasiswa06 mhsAwal = inputMahasiswa(scan);
            list.addFirst(mhsAwal);
            break;
        case 2:
            Mahasiswa06 mhsAkhir = inputMahasiswa(scan);
            list.addLast(mhsAkhir);
            break;
        case 3:
            System.out.print("Masukkan NIM yang dicari : ");
            String keyNim = scan.nextLine();
            System.out.println("Masukkan data baru: ");
            Mahasiswa06 dataBaru = inputMahasiswa(scan);
            list.insertAfter(keyNim, dataBaru);
            break;
        //case 4:
        //list.removeFirst();
        //break;
        //case 5:
        //list.removeLast();
        //break;
        case 6:
            list.print();
            break;
        case 0:
            System.out.println("Program selesai.");
            break;
        default:
            System.out.println("Menu tidak valid.");
    }
} while (pilihan != 0);
scan.close();
}
}

```

→ Hasil Running

===== MENU DOUBLE LINKED LIST =====

1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar

Pilih menu : 2

Masukkan NIM : 123005

Masukkan Nama : Harry

Masukkan Kelas : 1A

Masukkan IPK : 3,76

===== MENU DOUBLE LINKED LIST =====

1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar

Pilih menu : 3

Masukkan NIM yang dicari : 123005

Masukkan data baru:

Masukkan NIM : 123010

Masukkan Nama : Potter

Masukkan Kelas : 1B

Masukkan IPK : 3,55

Data berhasil disisipkan setelah NIM 123005

===== MENU DOUBLE LINKED LIST =====

1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar

Pilih menu : 6

NIM : 123005

Nama : Harry

Kelas : 1A

IPK : 3.76

NIM : 123010
Nama : Potter
Kelas : 1B
IPK : 3.55

Pertanyaan

1. Jelaskan perbedaan struktur dan mekanisme traversal antara Single Linked List dan Double Linked List!

Jawab: Single Linked List memiliki satu pointer yaitu next yang menunjuk ke node berikutnya sehingga traversal hanya dapat dilakukan dari depan ke belakang. Sedangkan Double Linked List memiliki dua pointer yaitu next dan prev, sehingga traversal dapat dilakukan dua arah, yaitu dari head ke tail maupun dari tail ke head.

2. Perhatikan class Node, di dalamnya terdapat atribut next dan prev. Jelaskan fungsi masing-masing atribut tersebut pada proses traversal dan manipulasi node!

Jawab: Atribut next berfungsi untuk menyimpan alamat node berikutnya sehingga proses traversal maju dapat dilakukan. Sedangkan atribut prev berfungsi menyimpan alamat node sebelumnya sehingga traversal mundur dapat dilakukan dan mempermudah proses penyisipan maupun penghapusan node.

3. Perhatikan konstruktor pada class DoubleLinkedList. Jelaskan fungsi konstruktor tersebut terhadap kondisi awal linked list!

Jawab: Konstruktor pada class DoubleLinkedList berfungsi menginisialisasi linked list dalam keadaan kosong dengan memberikan nilai null pada head dan tail. Dengan demikian, linked list belum memiliki node saat pertama kali dibuat.

4. Perhatikan potongan kode berikut:

```
if (isEmpty()) {  
    head = tail = newNode;  
}
```

Mengapa head dan tail harus menunjuk node yang sama ketika linked list masih kosong?

Jawab: Karena saat linked list kosong dan node pertama ditambahkan, node tersebut menjadi satu-satunya node yang ada di dalam linked list, sehingga node tersebut sekaligus berperan sebagai node pertama (head) dan node terakhir (tail). Jika hanya salah satu yang diset, misalnya hanya head = newNode tanpa tail = newNode, maka tail tetap bernilai null sehingga operasi seperti addLast() atau removeLast() yang bergantung pada tail dapat menyebabkan error.

5. Modifikasi method print() agar menampilkan pesan "Linked List masih kosong" ketika tidak terdapat data pada linked list!

Jawab:

```
public void print() {
    if (isEmpty()) {
        System.out.println(x: "Linked List masih kosong.");
        return;
    }
}
```

> Hasil Running

```
===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar
Pilih menu : 6
Linked List masih kosong.
```

6. Modifikasi kode program dengan menambahkan method `printReverse()` untuk menampilkan seluruh data pada Double Linked List secara terbalik, dimulai dari node tail menuju head!

Jawab:

→ Kode Program

> Modifikasi kode program class `DoubleLinkedList06.java`

```
public void printReverse() {
    if (isEmpty()) {
        System.out.println("Linked List masih kosong.");
        return;
    }

    Node06 current = tail;
    while (current != null) {
        System.out.println("-----");
        current.data.tampil();
        current = current.prev;
    }
}
```

> Modifikasi kode program class `DoubleLinkedListMain06.java`

```
- System.out.println("7. Tampilkan data reverse");
- case 7:
    list.printReverse();
    break;
```

→ Hasil Running

===== MENU DOUBLE LINKED LIST =====

1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
7. Tampilkan data reverse
0. Keluar

Pilih menu : 2

Masukkan NIM : 123005

Masukkan Nama : Harry

Masukkan Kelas : 1A

Masukkan IPK : 3,76

===== MENU DOUBLE LINKED LIST =====

1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
7. Tampilkan data reverse
0. Keluar

Pilih menu : 2

Masukkan NIM : 123010

Masukkan Nama : Potter

Masukkan Kelas : 1B

Masukkan IPK : 3,55

===== MENU DOUBLE LINKED LIST =====

1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
7. Tampilkan data reverse
0. Keluar

Pilih menu : 7

NIM : 123010

Nama : Potter

Kelas : 1B

IPK : 3.55

NIM : 123005

Nama : Harry

Kelas : 1A IPK : 3.76

12.3 Percobaan 2: Operasi Penghapusan pada Double Linked List

→ Kode Program

> Kode program class Mahasiswa06.java

```
package Jobsheet12;

public class Mahasiswa06 {
    String nim;
    String nama;
    String kelas;
    double ipk;

    public Mahasiswa06(String nim, String nama, String kelas, double ipk) {
        this.nim = nim;
        this.nama = nama;
        this.kelas = kelas;
        this.ipk = ipk;
    }

    public void tampil() {
        System.out.println(
            "NIM      : " + nim +
            "\nNama    : " + nama +
            "\nKelas  : " + kelas +
            "\nIPK     : " + ipk
        );
    }
}
```

> Kode program class Node06.java

```
package Jobsheet12;
public class Node06 {
    Mahasiswa06 data;
    Node06 prev;
    Node06 next;

    public Node06(Mahasiswa06 data) {
        this.data = data;
        this.prev = null;
        this.next = null;
    }
}
```

> Kode Program class DoubleLinkedList06.java

```
package Jobsheet12;
```

```

public class DoubleLinkedList06 {
    Node06 head;
    Node06 tail;

    public DoubleLinkedList06() {
        head = null;
        tail = null;
    }

    public boolean isEmpty() {
        return head == null;
    }

    public void addFirst(Mahasiswa06 data) {
        Node06 newNode = new Node06(data);
        if (isEmpty()) {
            head = tail = newNode;
        } else {
            newNode.next = head;
            head.prev = newNode;
            head = newNode;
        }
    }

    public void addLast(Mahasiswa06 data) {
        Node06 newNode = new Node06(data);
        if (isEmpty()) {
            head = tail = newNode;
        } else {
            tail.next = newNode;
            newNode.prev = tail;
            tail = newNode;
        }
    }

    public void insertAfter(String keyNim, Mahasiswa06 data) {
        Node06 current = head;
        while (current != null && !current.data.nim.equals(keyNim)) {
            current = current.next;
        }
        if (current == null) {
            System.out.println("Data dengan NIM " + keyNim + " tidak ditemukan.");
            return;
        }

        Node06 newNode = new Node06(data);

        if (current == tail) {
            newNode.prev = current;

```

```

        current.next = newNode;
        tail = newNode;
    } else {
        newNode.prev = current;
        newNode.next = current.next;
        current.next.prev = newNode;
        current.next = newNode;
    }
    System.out.println("Data berhasil disisipkan setelah NIM " + keyNim);
}

```

```

public void print() {
    if (isEmpty()) {
        System.out.println("Linked List masih kosong.");
        return;
    }
}

```

```

Node06 current = head;
while (current != null) {
    System.out.println("-----");
    current.data.tampil();
    current = current.next;
}
}

```

```

public void printReverse() {
    if (isEmpty()) {
        System.out.println("Linked List masih kosong.");
        return;
    }
}

```

```

Node06 current = tail;
while (current != null) {
    System.out.println("-----");
    current.data.tampil();
    current = current.prev;
}
}

```

```

public void removeFirst() {
    if (isEmpty()) {
        System.out.println("Linked List kosong.");
        return;
    }
    System.out.println("Data berhasil dihapus.");
    head.data.tampil();

    if (head == tail) {
        head = tail = null;
    } else {

```

```

        head = head.next;
        head.prev = null;
    }
}

public void removeLast() {
    if (isEmpty()) {
        System.out.println("Linked List kosong.");
        return;
    }
    System.out.println("Data berhasil dihapus.");
    tail.data.tampil();

    if (head == tail) {
        head = tail = null;
    } else {
        tail = tail.prev;
        tail.next = null;
    }
}
}

```

> Kode Program class DoubleLinkedListMain06.java

```

package Jobsheet12;
import java.util.Scanner;

public class DoubleLinkedListMain06 {
    public static Mahasiswa06 inputMahasiswa(Scanner scan) {
        System.out.print("Masukkan NIM : ");
        String nim = scan.nextLine();
        System.out.print("Masukkan Nama : ");
        String nama = scan.nextLine();
        System.out.print("Masukkan Kelas : ");
        String kelas = scan.nextLine();
        System.out.print("Masukkan IPK : ");
        double ipk = scan.nextDouble();
        scan.nextLine();
        return new Mahasiswa06(nim, nama, kelas, ipk);
    }

    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        DoubleLinkedList06 list = new DoubleLinkedList06();
        int pilihan;

        do {
            System.out.println("\n===== MENU DOUBLE LINKED LIST =====");
            System.out.println("1. Tambah data di awal");
            System.out.println("2. Tambah data di akhir");
            System.out.println("3. Sisipkan data di tengah (setelah NIM)");

```

```

System.out.println("4. Hapus data di awal");
System.out.println("5. Hapus data di akhir");
System.out.println("6. Tampilkan data");
System.out.println("7. Tampilkan data reverse");
System.out.println("0. Keluar");
System.out.print("Pilih menu : ");
pilihan = scan.nextInt();
scan.nextLine();

switch (pilihan) {
    case 1:
        Mahasiswa06 mhsAwal = inputMahasiswa(scan);
        list.addFirst(mhsAwal);
        break;
    case 2:
        Mahasiswa06 mhsAkhir = inputMahasiswa(scan);
        list.addLast(mhsAkhir);
        break;
    case 3:
        System.out.print("Masukkan NIM yang dicari : ");
        String keyNim = scan.nextLine();
        System.out.println("Masukkan data baru: ");
        Mahasiswa06 dataBaru = inputMahasiswa(scan);
        list.insertAfter(keyNim, dataBaru);
        break;
    case 4:
        list.removeFirst();
        break;
    case 5:
        list.removeLast();
        break;
    case 6:
        list.print();
        break;
    case 7:
        list.printReverse();
        break;
    case 0:
        System.out.println("Program selesai.");
        break;
    default:
        System.out.println("Menu tidak valid.");
}
} while (pilihan != 0);
scan.close();
}
}

```

→ Hasil Running

===== MENU DOUBLE LINKED LIST =====

1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
7. Tampilkan data reverse
0. Keluar

Pilih menu : 2

Masukkan NIM : 123005

Masukkan Nama : Harry

Masukkan Kelas : 1A

Masukkan IPK : 3,76

===== MENU DOUBLE LINKED LIST =====

1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
7. Tampilkan data reverse
0. Keluar

Pilih menu : 3

Masukkan NIM yang dicari : 123005

Masukkan data baru:

Masukkan NIM : 123010

Masukkan Nama : Potter

Masukkan Kelas : 1B

Masukkan IPK : 3,55

Data berhasil disisipkan setelah NIM 123005

===== MENU DOUBLE LINKED LIST =====

1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
7. Tampilkan data reverse
0. Keluar

Pilih menu : 6

NIM : 123005

Nama : Harry

Kelas : 1A

IPK : 3.76

NIM : 123010
Nama : Potter
Kelas : 1B
IPK : 3.55

===== MENU DOUBLE LINKED LIST =====

1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
7. Tampilkan data reverse
0. Keluar

Pilih menu : 5

Data berhasil dihapus.

NIM : 123010
Nama : Potter
Kelas : 1B
IPK : 3.55

===== MENU DOUBLE LINKED LIST =====

1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
7. Tampilkan data reverse
0. Keluar

Pilih menu : 6

NIM : 123005
Nama : Harry
Kelas : 1A
IPK : 3.76

Pertanyaan

1. Perhatikan potongan kode berikut pada method removeFirst():

```
head = head.next;  
head.prev = null;
```

Jelaskan fungsi masing-masing statement tersebut pada proses penghapusan node!

Jawab: Pada proses penghapusan node pertama pada Double Linked List, statement `head = head.next` berfungsi untuk memindahkan pointer head ke node berikutnya sehingga node pertama tidak lagi menjadi bagian dari linked list. Selanjutnya, statement

head.prev = null berfungsi untuk memutus hubungan antara node baru yang menjadi head dengan node sebelumnya yang telah dihapus sehingga struktur linked list tetap benar.

2. Modifikasi method removeFirst() dan removeLast() agar program menampilkan data yang berhasil dihapus!

Jawab:

```
public void removeFirst() {  
    if (isEmpty()) {  
        System.out.println(x: "Linked List kosong.");  
        return;  
    }  
  
    System.out.println(x: "Data berhasil dihapus.");  
    head.data.tampil();  
  
    if (head == tail) {  
        head = tail = null;  
    } else {  
        head = head.next;  
        head.prev = null;  
    }  
}
```

```
public void removeLast() {  
    if (isEmpty()) {  
        System.out.println(x: "Linked List kosong.");  
        return;  
    }  
  
    System.out.println(x: "Data berhasil dihapus.");  
    tail.data.tampil();  
  
    if (head == tail) {  
        head = tail = null;  
    } else {  
        tail = tail.prev;  
        tail.next = null;  
    }  
}
```

> Hasil Running

===== MENU DOUBLE LINKED LIST =====

1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
7. Tampilkan data reverse
0. Keluar

Pilih menu : 5

Data berhasil dihapus.

NIM : 123010

Nama : Potter

Kelas : 1B

IPK : 3.55